Comenius University in Bratislava
Faculty of Mathematics, Physics and Informatics

Machine Learning for Generation of Graph of Given Degree and Girth

Bachelor Thesis

2026
Vladimír Jančár

Comenius University in Bratislava
Faculty of Mathematics, Physics and Informatics

Machine Learning for Generation of Graph of Given Degree and Girth

Bachelor Thesis

Study Programme: Applied Computer Science
Field of Study: Computer Science
Department: Department of Applied Informatics
Supervisor: Mgr. Ján Pastorek

Bratislava, 2026
Vladimír Jančár

Acknowledgments:

This section will be completed near the end of the thesis.

1 Abstract

This thesis investigates machine-learning methods for generating graphs of prescribed degree and girth, with emphasis on graph neural networks and constrained search for (k, g)-graphs. The final abstract will summarize the problem, proposed method, experiments, and main findings once the experimental contribution is fixed.

Keywords: graph neural networks, algebraic graph theory, cage problem, (k, g)-graphs, voltage graph lifts, heuristic search

Contents

List of Figures

List of Tables

2 Introduction

Temporary purpose of this chapter. This chapter should be written after the core contribution is clear, but before the abstract and conclusion. It should introduce the cage problem, explain why constructing small (k, g)-graphs is difficult, and state the thesis goal: using graph neural networks as learned heuristics inside constrained graph-construction search.

This thesis concerns machine-learning methods for generating graphs of prescribed degree and girth. In graph-theoretic language, the central objects are (k, g)-graphs: k -regular graphs whose shortest cycle has length at least g. The classical cage problem asks for such graphs with the smallest possible order. Known approaches include algebraic constructions, voltage graph lifts, exhaustive search, heuristic search, and excision techniques [1], [2].

The working hypothesis of this thesis is that graph neural networks are most useful not as unrestricted graph generators, but as learned components inside constrained search procedures. Regularity and girth can often be enforced, checked, or pruned directly, while a learned model can score partial states, voltage assignments, or local graph edits.

The official thesis goal is broader than one implementation strategy: it asks whether graph neural networks can be useful in algebraic graph theory, either by helping generate new record graphs or by learning to predict algebraic graph properties. This motivates the structure of the work. Preparatory prediction tasks test whether the models learn relevant graph properties, while the construction experiments investigate whether learned models can guide the search for graphs of given degree and girth.

To write later. Add the motivation, exact research questions, thesis contribution, and a paragraph describing the final chapter structure.

3 Background and Definitions

Temporary purpose of this chapter. This should be one of the first substantial chapters to draft. Definitions are the foundation for the rest of the thesis, and writing them early exposes notation problems before experiments and results are described.

3.1 Graphs

A graph is a pair $G = (V, E)$, where V is a finite set of vertices and E is a set of edges. In the main cage-construction setting, graphs are simple, undirected graphs unless explicitly stated otherwise.

For a vertex $v \in V$, the degree of v, denoted $\deg(v)$, is the number of edges incident with v. The open neighborhood of v, denoted $N(v)$, is the set of vertices adjacent to v.

3.2 Regular Graphs

A graph is k-regular if every vertex has degree k. Regularity is one of the two hard constraints in the (k, g)-graph construction task.

3.3 Walks, Paths, and Cycles

A walk is a sequence of vertices and edges in which consecutive vertices are adjacent. A path is a walk with no repeated vertices. A cycle is a closed path of length at least 3.

The girth of a graph, denoted $\text{girth}(G)$, is the length of its shortest cycle. If a graph has no cycles, its girth is sometimes treated as infinite.

3.4 (k, g)-Graphs and Cages

A (k, g)-graph is a k-regular graph whose girth is at least g. Some authors use girth exactly g when discussing cages; for construction algorithms, requiring girth at least g is the safer operational condition. A (k, g)-cage is a (k, g)-graph with the smallest possible number of vertices. This minimum order is commonly denoted $n(k, g)$.

The Moore bound gives a general lower bound on the order of a (k, g)-graph. It is useful both as a theoretical benchmark and as a practical target size for construction algorithms.

For $k \geq 2$ and $g \geq 3$, the Moore bound is

$$M(k,g) = \begin{cases} 1 + k\sum_{i=0}^{\frac{g-3}{2}} (k-1)^i \\ \text{if } \ g \ \text{ is odd} \\ 2\sum_{i=0}^{\frac{g-2}{2}} (k-1)^i \\ \text{if } \ g \ \text{ is even} \end{cases}.$$

A graph meeting this bound is called a Moore graph. Moore graphs are rare, which is one reason the cage problem remains difficult for most parameter pairs.

3.5 Graph Neural Networks

Ordinary feed-forward neural networks expect fixed-size vectors. Graphs do not have a fixed number of vertices, and their vertices do not have a canonical ordering. Graph neural networks address this by applying the same local update rule at every vertex. This makes the model independent of the input graph size and equivariant to relabeling of vertices.

Message-passing graph neural networks process graph-structured data by iteratively updating vertex representations through layers [3]. At layer t, each vertex v has a hidden representation h_v^t. The layer aggregates messages from the neighborhood $N(v)$ and then updates the representation:

$$m_v^t = \text{AGG}(\{h_u^{t-1} : u \in N(v)\})$$

$$h_v^t = \mathrm{UPD}(h_v^{t-1}, m_v^t).$$

Stacking several layers lets information travel farther through the graph: after one layer, a vertex representation depends on its immediate neighbors; after two layers, it can depend on vertices at distance two; and so on. This is why local quantities such as degree are easier for standard GNNs than global quantities such as girth.

Grohe's survey on the logic of graph neural networks provides the theoretical framing for this limitation [4]. Standard message-passing GNNs are closely related to color refinement and the one-dimensional Weisfeiler–Leman algorithm. This connection is important for the thesis because it explains why some graph properties are naturally accessible to ordinary GNNs while other global or symmetry-sensitive properties may require stronger architectures, additional features, or a different formulation of the search problem.

The project uses several GNN architectures:

- GCN, a stable baseline based on degree-normalized neighbor aggregation [5].
- GraphSAGE, an inductive architecture based on learned aggregation [6].
- GIN, a sum-aggregation architecture with expressivity related to the Weisfeiler–Leman test [7].
- GPS, a graph transformer architecture combining local message passing with global attention [8].
- Loopy GNN, a cycle-aware architecture relevant to minimum-cycle and girth-related tasks [9].

3.6 Voltage Graph Lifts

A voltage graph construction starts with a small base graph, a finite group, and group-element labels on the directed edges. The corresponding lift expands the base graph into a larger graph. If the base graph is k-regular, the lifted graph is also k-regular.

This representation is useful for cage construction because it enforces regularity structurally and shifts the search problem toward choosing good voltage assignments.

To write later. Add formal voltage-lift notation and the net-voltage characterization of girth used by `ai/cage/voltage/cycle_analysis.py`.

4 Related Work

Temporary purpose of this chapter. This chapter should be drafted after the core definitions are stable. It should explain what classical graph theory and computational search already provide, so the thesis contribution is positioned honestly.

The cage problem has been studied through algebraic constructions, finite geometries, Cayley graphs, voltage graph lifts, exhaustive generation, local search, and excision.

The Dynamic Cage Survey provides the broad background and record-holder context [1].

Recent computational work is especially relevant to this thesis. Exoo et al. describe lift-based generation, pruning of voltage assignments, tabu search, hill climbing, and excision methods for improving upper bounds on cage and related graph-construction problems [2]. Exoo also studies order-decreasing transformations and graph-distance moves that preserve regularity and girth or make excision possible [10].

The machine-learning side of the project should be placed against work on graph neural networks and learned heuristics for combinatorial search, not only against graph generation papers. The relevant question is whether a learned model can guide a hard discrete search procedure. The reviewer feedback on the earlier SVK paper pointed to learned heuristics for SAT solvers as a useful analogy: the learned component need not solve the problem alone, but can replace or improve a branching or scoring heuristic inside a deterministic algorithm.

To write later. Separate this into subsections: finite-geometry/algebraic constructions, voltage graph lifts, exhaustive and heuristic search, excision, graph generation with reinforcement learning, neural combinatorial optimization, and expressive GNNs beyond 1-WL. Be careful to cite every construction and not overstate novelty.

5 Preparatory GNN Tasks

Status of this chapter. This chapter describes work that is unlikely to change conceptually. The final numerical tables may still change, but the motivation for these tasks and their role in the thesis are stable.

Before attempting graph generation, the project first evaluated several GNN architectures on two node-level prediction tasks: degree prediction and minimum-cycle prediction. These tasks are not the final goal of the thesis. They serve as controlled probes for the two structural constraints that define a (k, g)-graph.

Degree prediction is local and exactly verifiable. A vertex degree can be recovered from the immediate neighborhood, so this task tests whether the model, data pipeline, and training loop can learn a simple structural quantity. If an architecture fails to predict degree reliably, it is not a good candidate for harder construction tasks without further tuning.

Minimum-cycle prediction is a harder structural task. For each vertex, the target is the length of the shortest cycle containing that vertex, with target 0 for vertices that do not lie on any cycle. Unlike degree, this cannot be determined from immediate neighbors alone. It requires information about paths that leave a vertex and later return to it. This makes the task closer to the girth constraint used in cage construction.

5.1 Data Generation

Both tasks use dynamically generated random graphs. Instead of training on a fixed set of graphs, new Erdos–Renyi graphs are sampled during training. This reduces the chance that a model simply memorizes a finite training set and gives a cheaper way to expose it to many small graph structures.

The current implementation generates graph labels directly from exact algorithms. Degree labels are computed as $\deg(v)$ for each vertex. Minimum-cycle labels are computed by searching for the smallest cycle containing each vertex. This keeps the supervision reliable even though the graphs themselves are random.

The input features are intentionally simple. The code supports either a constant one-dimensional feature or a four-dimensional feature vector consisting of a normalized node index, two random real-valued coordinates, and one additional random scalar. These features should not be interpreted as meaningful graph attributes. They mainly give the neural network a vertex-level input tensor while forcing the model to learn from graph structure.

5.2 Architectures Compared

The same family of architectures is used across the preparatory tasks: GCN, GraphSAGE, GIN, GPS, and Loopy GNN. This comparison is useful because the architectures have different inductive biases. GCN normalizes neighbor messages by degree, which can make raw counting harder. GIN and GraphSAGE preserve count-like information more directly. GPS adds global attention, and Loopy adds explicit cycle-aware information.

5.3 Evaluation

The early experiments used a regression objective with mean squared error and evaluated by rounding predictions to the nearest integer. This produces an intuitive exact-node accuracy, but it is a brittle metric: a prediction off by one receives no credit. For the thesis, exact accuracy should therefore be reported together with mean absolute error and mean squared error. If the task is later reframed as classification over integer labels, confusion matrices should also be reported.

5.4 Lessons From the First Experiments

The first small-capacity experiments were useful, but they should not be treated as final evidence about the limits of the architectures. A reviewer of the earlier SVK paper correctly pointed out that hidden dimension 32 with four layers is an underpowered setting for strong architectural claims. The correct conclusion is narrower: in the initial configuration, GIN and GraphSAGE were the most reliable for degree prediction, while minimum-cycle prediction remained difficult and exposed the need for larger models or more cycle-aware architectures.

Later project documentation indicates that increasing capacity changes the picture, especially for Loopy and GPS. This supports a more cautious thesis framing: the preparatory tasks identify promising model families and failure modes, but they do not by themselves solve the cage-construction problem.

The main value of this chapter is therefore methodological. It explains why the project did not jump directly into graph generation. Degree prediction tested basic structural learning, while minimum-cycle prediction tested whether the models could learn information related to girth.

6 Cage Construction Methods

Status of this chapter. This chapter records the development path toward the current construction approach. It should be written honestly: the early attempts are important because they explain why the project moved toward constrained search and voltage lifts.

The target problem is to construct small (k, g)-graphs. This is harder than predicting a graph invariant, because the algorithm must actively choose edges while preserving regularity and avoiding short cycles. The search space grows quickly with the number of vertices, and most arbitrary edge choices lead to invalid or unpromising graphs.

The problem is therefore more naturally viewed as a search problem than as a single prediction problem. A construction algorithm repeatedly holds a partial object, chooses a modification, checks which constraints are still satisfiable, and continues until it either reaches a valid graph or enters an unpromising region of the search space. Machine learning can enter this process in several different ways: as a learned heuristic for deterministic search, as a policy trained from complete construction attempts, or as a scoring function inside a constrained algebraic search.

6.1 Guided Search as the First Idea

A natural first idea is to use deterministic search and let a GNN guide it. For example, one can build a graph edge by edge and use a learned scoring function to prioritize partial graphs that appear more likely to extend to a valid (k, g)-graph.

The difficulty is not the search loop itself. The difficulty is obtaining useful labels for partial states. Positive examples can be obtained by taking subgraphs of known valid graphs. Useful negative examples are harder: a partial graph may look bad but still be extendable after adding many more edges. Deciding whether a partial graph can be completed into a valid (k, g)-graph is itself a hard search problem. As a result, the supervised version risks training on easy examples far from the decision boundary, which gives little useful guidance to the search.

This explains why the project moved away from a purely supervised A-star-style approach. The obstacle is not that guided search is unnatural; on the contrary, it is a

natural formulation. The obstacle is that the labels needed to train a good heuristic already require solving a difficult completion problem.

6.2 Why Reinforcement Learning Was Tried

Reinforcement learning is one standard way to train a policy for a sequential decision problem when reliable intermediate labels are unavailable. Instead of asking for a dataset that says whether each partial graph is extendable, the model learns from complete construction attempts. The environment supplies rewards based on the observed consequences of actions: whether an edge edit was valid, whether the graph moved closer to regularity, whether short cycles were avoided, and whether the final graph satisfies the target constraints.

In this sense, the reinforcement-learning approach is not separate from search. It is a learned version of heuristic search. A hand-designed search algorithm chooses moves according to a manually specified rule; an RL policy attempts to learn such a rule from repeated interaction with the construction environment. This makes it a reasonable first learned-construction method after the supervised heuristic approach becomes difficult to label.

At the same time, this choice should be interpreted cautiously. Direct reinforcement learning does not build in much graph-theoretic structure. The agent must learn which vertices should become active, which edges should be added or removed, how to satisfy degree constraints, and how to avoid short cycles. The experiment is therefore useful partly as a diagnostic: it tests how far an unconstrained learned search formulation can go before stronger mathematical structure has to be inserted into the representation.

6.3 Direct Reinforcement Learning

The next approach reframed construction as a sequential decision problem and used Proximal Policy Optimization [11]. The environment represents a partially constructed graph. Each action corresponds to an unordered pair of vertices. If the edge is absent, the action attempts to add it; if the edge is present, the action attempts to remove it.

This direct graph-editing formulation has an advantage: it does not require a precomputed dataset of labeled partial graphs. The model learns from interaction. However, it also makes reward design and action pruning essential.

Several invalid or unhelpful actions can be ruled out directly:

- An edge is not added if either endpoint already has degree k.
- An edge is not added if it would create a cycle shorter than g. If adding edge (u, v) creates a new short cycle, that cycle must include the new edge, so it is enough to check the shortest path between u and v before the edge is added.
- An edge is not removed if doing so disconnects the active part of the graph.

- An edge is not removed if too few active vertices would remain to reach the Moore-bound target.

The initial reward was sparse: small rewards for adding edges, penalties for invalid or removing actions, and a large terminal reward for constructing a valid graph. This was not enough. The agent often reached dead-end states and received little directional signal about how to improve.

The later reward added progress-based shaping. A potential function measured partial progress toward the target, combining degree regularity and edge count. The reward then included the change in potential after each action. This made intermediate improvements visible to the agent.

One important lesson was that the usual discounted potential-shaping expression can behave badly in long unfinished episodes. With discount factor $\gamma < 1$, maintaining a high-potential but incomplete graph creates an implicit per-step penalty. In this project, the more practical choice was the undiscounted potential difference $\Phi(s') - \Phi(s)$.

6.4 Curriculum Learning

The environment samples parameter pairs in increasing difficulty, ordered by the Moore bound. Training starts with small cases such as $(3, 5)$ and $(3, 6)$. A new stage is unlocked only after the agent reaches a required success rate over a recent window of episodes. This curriculum avoids starting immediately on parameter pairs where successful episodes are too rare to provide a useful learning signal.

6.5 Limits of the Direct Formulation

The direct graph-editing formulation is useful for experimentation, but it has structural weaknesses. Regularity is not guaranteed; it must be learned or enforced through action pruning and rewards. The action space also grows quadratically with the number of available vertices. Finally, success on small cages does not by itself imply that the method scales to harder open cases.

For this reason, results from the direct PPO environment should be reported as exploratory unless they are compared against strong baselines such as random search, greedy search, brute force on small instances, or known construction methods. Its main role in this thesis is methodological: it shows that learning from interaction is possible, but that too much capacity is spent rediscovering constraints that are already known from graph theory.

6.6 Transition to Voltage Lifts

The later direction uses voltage graph lifts. This is not just a different neural-network architecture; it is a different representation of the construction problem. The search starts with a small base graph and a finite group. The algorithm assigns

group elements, called voltages, to the base graph edges. The lift then produces a larger graph.

This representation is attractive because regularity is built into the construction: if the base graph is k-regular, the lift is also k-regular. The learning problem can therefore focus more directly on avoiding short cycles and choosing promising voltage assignments.

Thus the transition from direct RL to voltage lifts is not a change from one arbitrary experiment to another. It is a response to the weaknesses of the first formulation. The direct environment made the construction problem as general as possible; the voltage formulation instead uses known algebraic structure to remove regularity from the learning problem and to provide a cheaper exact signal for girth violations.

The current codebase includes exact girth computation for voltage lifts using net-voltage analysis, random search, exhaustive search for small groups, tabu search, GNN-guided beam search, meta-search over base graphs and groups, and a reinforcement-learning environment for voltage assignment. This formulation also aligns better with recent non-AI cage-construction work, where pruning, tabu search, hill climbing, and excision are applied inside highly constrained search spaces [2].

To write later. Keep this chapter as the development path. The detailed explanation of voltage lifts belongs in the next chapter.

7 Voltage Graph Lifts

Reason for this chapter. This chapter should become the main explanation of the current promising approach. It deserves to stand on its own because it changes the problem formulation, not just the model architecture.

The direct graph-editing environment asks the learning algorithm to satisfy two difficult constraints at the same time: every vertex should have degree k, and the graph should avoid all cycles shorter than g. Voltage graph lifts remove one of these burdens. If the base graph is k-regular, then every lift of that base graph is also k -regular. The search therefore shifts from constructing all edges of a large graph to choosing group labels on the edges of a much smaller base graph.

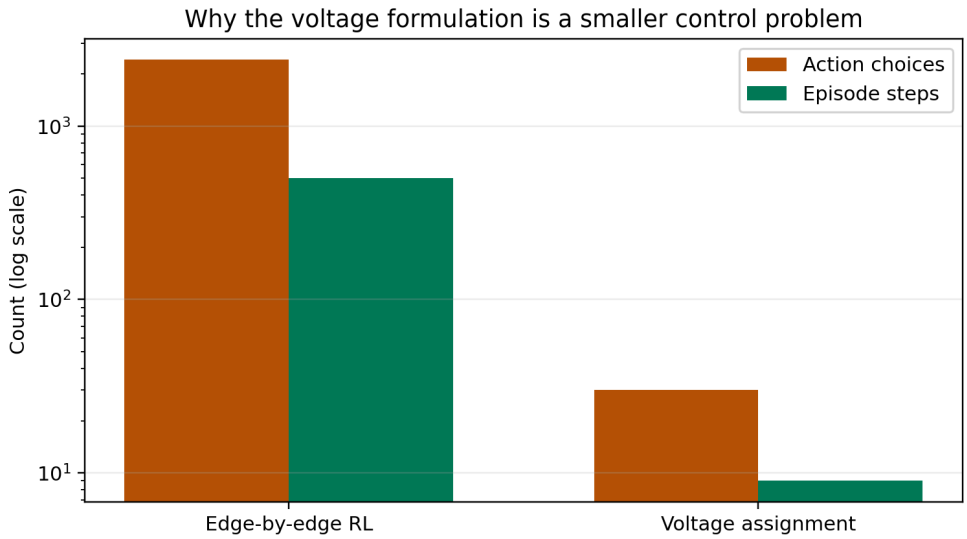

Figure 1: The voltage formulation has a much smaller control problem than direct edge-by-edge graph editing. The numbers shown are illustrative: a 70-vertex direct environment has thousands of possible edge actions, while a voltage assignment step chooses one group element.

7.1 Construction

A voltage-lift construction uses three ingredients:

- a small base graph B,
- a finite group Γ,
- a voltage assignment α that maps every directed edge of B to an element of Γ.

For undirected graphs, each edge is represented by two opposite directed arcs. If one direction has voltage a, then the reverse direction receives a^{-1}. The lift has vertex set $V(B) \times \Gamma$. For an arc from u to v with voltage a, the lift connects

$$(u, h) \quad \text{to} \quad (v, ha)$$

for every group element $h \in \Gamma$.

In the codebase, this construction is implemented in `ai/cage/voltage/lift.py`. A vertex pair (u, h) is encoded as the integer `u * group.order + h`. The resulting graph has $|V(B)| \cdot |\Gamma|$ vertices. The verification step checks connectedness, k-regularity, and girth.

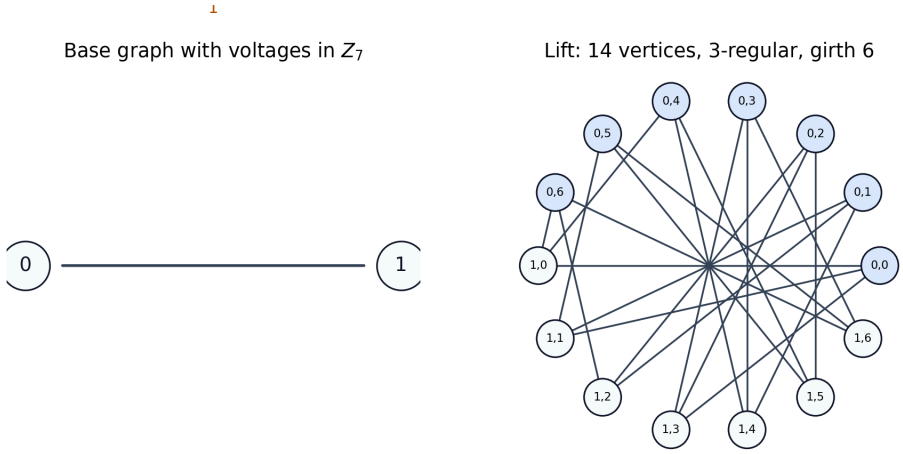

Figure 2: A two-vertex base graph with three parallel edges and voltages in Z_7 lifts to a 14-vertex cubic graph. With a suitable assignment, this produces the Heawood graph, the $(3, 6)$-cage.

7.2 Why This Helps

The most important property is preservation of regularity. In direct graph editing, the agent must learn to keep every vertex near degree k. In the voltage formulation, this is structural. A k-regular base graph produces a k-regular lift automatically. This is why the approach is a better match for cage construction than unrestricted graph generation.

The second advantage is that girth can be analyzed on the base graph. The girth of the lift is determined by closed non-backtracking walks in the base graph whose net voltage is the identity element of the group. Therefore, short-cycle violations can be detected without constructing the full lifted graph. The implementation in `ai/cage/voltage/cycle_analysis.py` enumerates short closed walks in the base graph and multiplies voltages along the walk.

This gives a cheap exact cost for search:

- if a short closed walk has identity net voltage, it corresponds to a short cycle in the lift;
- if no such walk exists below length g, the lift has girth at least g up to the checked bound.

7.3 Search Methods in the Current Framework

The voltage framework currently contains several search strategies.

Exhaustive search. For small groups and few base edges, the code enumerates every voltage assignment. This is only practical when $|\Gamma|^m$ is small, where m is the number of undirected base edges.

Random search. Random voltage assignments are sampled and verified. This is simple, but it is a useful baseline because some small cases are easy enough that random search succeeds.

Tabu search. The tabu search implementation follows the recent computational cage-construction literature [2]. It fixes tree-edge voltages to the identity, so only non-tree edges are searched. It then changes one voltage at a time and minimizes the number of short identity-voltage walks. A tabu list prevents the algorithm from immediately undoing recent moves.

GNN-guided beam search. The beam-search implementation assigns voltages edge by edge. A trained `GirthPredictor` scores partial assignments, and the search keeps only the highest-scoring candidates. The final candidates are still verified by exact girth computation.

Meta-search. The `meta_search` function iterates over candidate base graphs and finite groups. For cubic graphs, the candidates include the dumbbell base, a four-node cubic base, the prism base, and a Petersen-like base. Candidate groups include cyclic groups, dihedral groups, direct products, and semidirect products.

Voltage-assignment PPO. The reinforcement-learning environment in `ai/cage/voltage/rl_env.py` treats one voltage assignment as one episode. The state is the base graph with partial voltage labels, the action is a group element, and the episode length is the number of base edges. This is much shorter than the direct edge-editing environment. The model in `ai/cage/voltage/rl_model.py` uses a GINE/GPS encoder, global pooling, an actor head over group elements, and a critic head for PPO.

7.4 Current Interpretation

The important thesis claim should be modest. Voltage lifts are not new, and many hand-crafted or computational lift searches already exist. The contribution of this project should not be described as inventing voltage lifts. The relevant question is whether graph neural networks can guide this constrained search space better than simple baselines.

This framing also makes negative results more useful. If the learned component does not outperform random search, tabu search, or beam search, that is still informative: it shows that the chosen learned representation did not capture enough reusable structure. If it does outperform them on some configurations, the result is stronger because the comparison is against methods that already respect the mathematics of the problem.

8 Experiments and Results

Temporary purpose of this chapter. This should be written late, after experiments are stable. It should report final numbers, not exploratory anecdotes, and it should compare against non-learned baselines.

The current project documentation reports that PPO-based cage generation improved after reward shaping and larger models. A GPS model with hidden dimension 128, eight attention heads, and GIN convolution is currently described as the strongest direct graph-editing model, with nontrivial success rates on early curriculum stages.

For the final thesis, the results should answer whether the learned component improves construction search. The most important comparisons are likely:

- random search versus learned search,
- brute force on small cases where exhaustive search is possible,
- beam search versus GNN-guided beam search,
- tabu or hill-climbing baselines if implemented,
- direct graph editing versus voltage-assignment search.

To write later. Add final tables only after rerunning or verifying metrics. Avoid using stale documentation numbers as final thesis claims unless they are tied to exact model IDs, checkpoints, and evaluation scripts.

9 Conclusion

Write this last. The conclusion should be written after all experiments and result tables are final. It should state whether the thesis goal was achieved, summarize the main findings, describe limitations, and give concrete future work.

The final conclusion will summarize what was implemented and evaluated, whether graph neural networks provided useful guidance for constructing (k, g)-graphs, and which search formulation proved most promising.

To write later. Do not introduce new literature or new experiments here. The conclusion should only synthesize what the thesis has already shown.

Bibliography

[1] G. Exoo and R. Jajcay, "Dynamic Cage Survey," *The Electronic Journal of Combinatorics*, 2013.

[2] G. Exoo, J. Goedgebeur, J. Jooken, L. Stubbe, and T. Van den Eede, "New small regular graphs of given girth: the cage problem and beyond." 2025.

[3] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl, "Neural Message Passing for Quantum Chemistry," in *Proceedings of the 34th International Conference on Machine Learning*, 2017.

[4] M. Grohe, "The Logic of Graph Neural Networks." 2022.

[5] T. N. Kipf and M. Welling, "Semi-Supervised Classification with Graph Convolutional Networks," in *International Conference on Learning Representations*, 2017.

[6] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive Representation Learning on Large Graphs," in *Advances in Neural Information Processing Systems*, 2017.

[7] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How Powerful are Graph Neural Networks?," in *International Conference on Learning Representations*, 2019.

[8] L. Rampášek, M. Galkin, V. P. Dwivedi, A. T. Luu, G. Wolf, and D. Beaini, "Recipe for a General, Powerful, Scalable Graph Transformer," in *Advances in Neural Information Processing Systems*, 2022.

[9] R. Paolino, C. Vignac, and A. Loukas, "Weisfeiler and Leman Go Loopy: A New Hierarchy for Graph Representational Learning," in *International Conference on Learning Representations*, 2024.

[10] G. Exoo, "On decreasing the orders of (k, g)-graphs", *Journal of Combinatorial Optimization*, 2023, doi: 10.1007/s10878-023-01092-9.

[11] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," *arXiv preprint arXiv:1707.06347*, 2017.